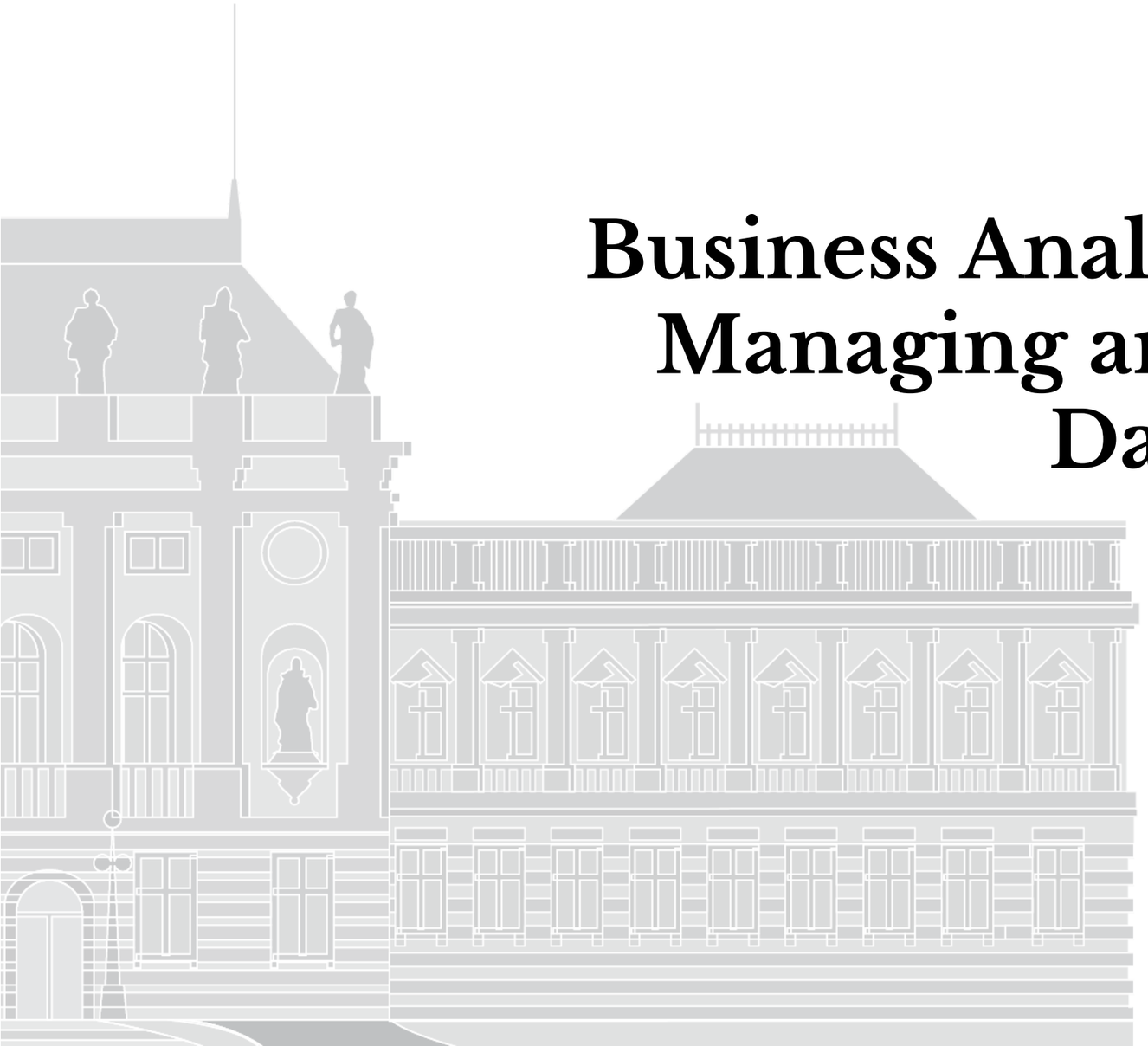
# Business Analytics with R – Managing and Cleaning Data



# Outline

01

## Goals of this lecture

02

## Base Functions vs External Packages

03

## Data Management with tidyverse

04

## Data Cleaning

# Goals of the current lecture

Students can clean and prepare data

# Base Functions vs External Packages

## Base Functions

- The basic R installation package comes with a handful of functions already installed.
- Those built-in functions are functions that you will encounter frequently when using R.
- We already used such base functions, e.g. `mean()`

## Functions from external packages

- You have the option to add more functions from external sources by installing external "packages". Those packages are (usually) available free of charge.
  - A list of all certified packages can be found on the R website (https://cran.r-project.org/web/packages/index.html).
- Anyone can write an R package and publish it. Most external packages are written by university employees or practitioners from a variety of industries.
- External packages cover a wide range of topics
  - Machine Learning, Advanced statistical analysis
  - Economic forecasting, Financial analysis, or
  - Data management

# Data Management with tidyverse

## The Tidyverse

The tidyverse is a coherent system of packages for data manipulation, exploration and visualization that share a common design philosophy.

### Consistent functions

### Workflow coverage

### Parsimonious approach

to the development of data science tools

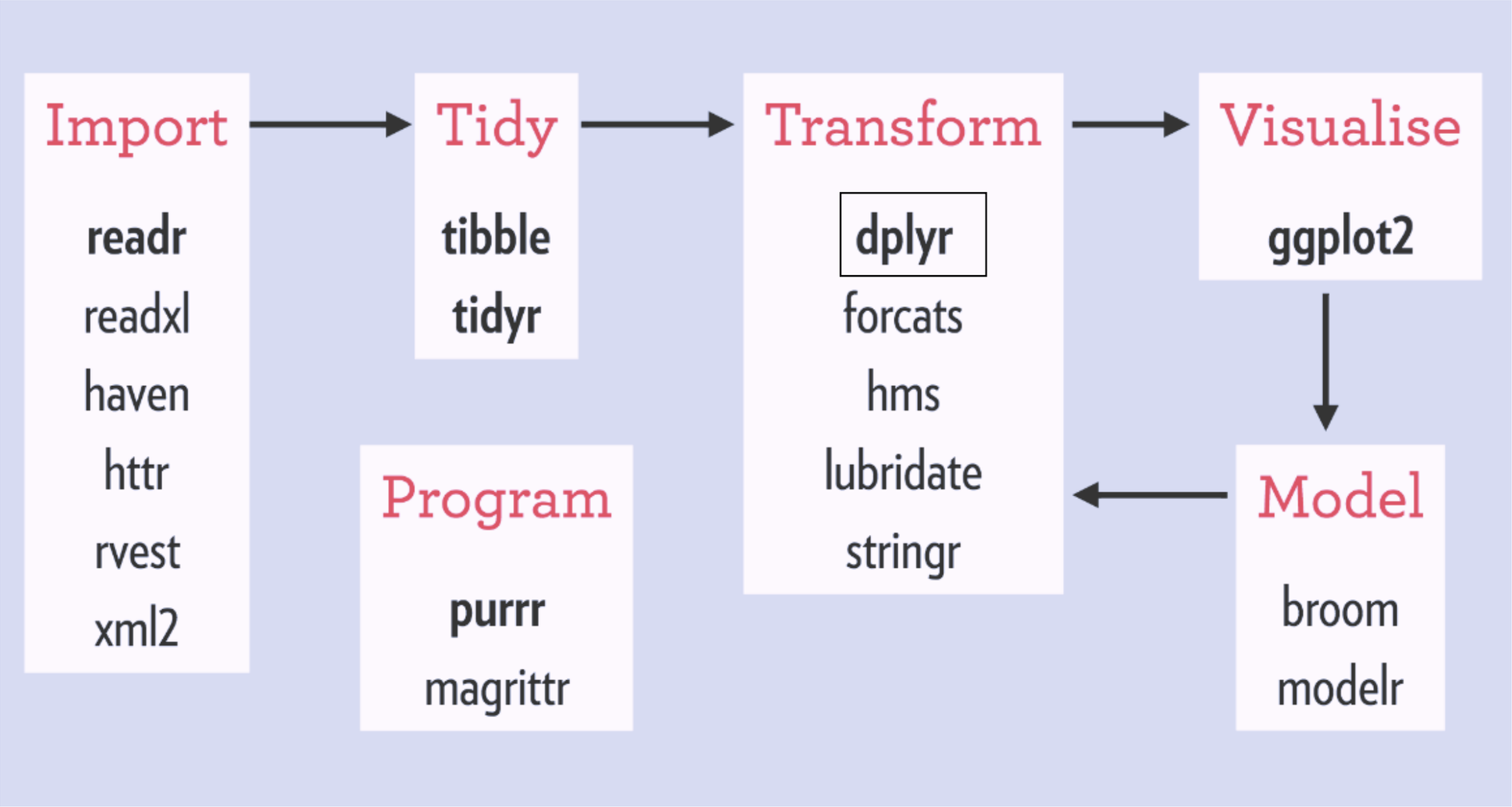
### Greater productivity

the possibility of greater productivity

```
install.packages("tidyverse")
```

# Tidyverse

The diagram illustrates the Tidyverse workflow, organized into five main stages: Import, Tidy, Transform, Visualise, and Model. Each stage is represented by a box containing specific R packages.



# The dplyr-package

## What is dplyr?

dplyr is a new package which provides a set of tools for efficiently manipulating datasets in R

## Why dplyr?

### Speed

dplyr is faster than many alternatives.

### Flexibility

dplyr can be used to manage data stored in many different sources (i.e. Excel- and CSV files, SQL databases,...)

### Cleaner Code

dplyr makes it easy to write clean code.

### Simpler Code

The code is easy to write and to follow.

# Pipes

## What is a pipe?

- Pipes let you pass an intermediate result onto the next function.
- Real Estate Agent:
  - Filter all houses with 2 bedrooms and compute the average price of the home.
  - `Real_estate_data %>%`
    `filter(bedrooms==2) %>%`
    `summarize(mean.prize = mean(prize))`

## Why use pipes?

- You'll structure the sequence of your data operations from left to right, as apposed to from inside and out;
- You'll avoid nested function calls;
- You'll minimize the need for local variables and function definitions; and
- You'll make it easy to add steps anywhere in the sequence of operations.

➔ Your code becomes easier to read, debug and less prone to errors

# Pipes – How can I use pipes?

"%>%" operator passes the outputs of one function as an input into the next function in the pipe.

```
# Initialize `x`
> x <- c(0.109, 0.359, 0.63, 0.996, 0.515, 0.142, 0.017, 0.829, 0.907)

# Compute the logarithm of `x`, return suitably lagged and iterated
differences,
# compute the exponential function and round the result
> round(exp(diff(log(x))), 1)

# Import `magrittr`
> library(magrittr)

# Perform the same computations on `x` as above
> x %>%
> log() %>%
> diff() %>%
> exp() %>%
> round(1)
```

## dplyr::filter

Sometimes you only want to look at observations with certain characteristics (i.e. homes that were newly constructed, or homes with electric heating systems). You can filter these observations with the command „`filter(conditions)`"

```
# Import data from housing dataset
housing_data <- read.csv('Housing_Prices.csv',header=TRUE,sep=';')

# Import `dplyr`
library(dplyr)

# Filter new homes with electric heating and store them in a new
dataset
housing_data_electric <- housing_data %>%
  filter(Fuel.Type=="Electric")

#Looking at the new dataset
print(housing_data_electric)
```

| Operator | Description |
|---|---|
| `<` | less than |
| `<=` | less than or equal to |
| `>` | greater than |
| `>=` | greater than or equal to |
| `==` | exactly equal to |
| `!=` | not equal to |
| `x \| y` | x OR y |
| `x & y` | x AND y |

# dplyr::select and dplyr::mutate

## dplyr::select

You can also remove certain variables/columns from a dataframe using the command „select"

```
# Import data from housing dataset
housing_data <-
read.csv('Housing_Prices.csv',header=TRUE,sep=';')

# Import `dplyr`
library(dplyr)

# Remove all attributes except for Price and Age
housing_data_reduced <- housing_data %>%
  select(Price, Age)

#Looking at the new dataset
print(housing_data_reduced)
```

## dplyr::mutate

`mutate()` computes a new variables

```
# Import data from housing dataset
housing_data <-
read.csv('Housing_Prices.csv',header=TRUE,sep=';')

# Import `dplyr`
library(dplyr)

# Calculate price per square metre
housing_data <- housing_data %>% #store all changes
below in the same dataset!
 mutate(price_sqft = Price / Living.Area)

print(housing_data)
```

# Data Cleaning – Messy Dataset

A real-world dataset often contains many issues. Below is an example of a messy dataset (`head(df)`):

| regio1 | serviceCharge | heatingType | telekomtvOffer | cid | newlyConst | balcony |
|---|---|---|---|---|---|---|
| Sachsen_Anhalt | 120 | central_heating | ONE_YEAR_FREE | 796687 | n | y |
| Sachsen | 390 | central_heating | ONE_YEAR_FREE | 226664 | n | y |
| Hessen | 380 | gas_heating | ONE_YEAR_FREE | 17271157 | n | y |
| Berlin | 200 | combined_heat_and_power_plant | ONE_YEAR_FREE | 17720196 | n | y |
| Sachsen | 150 | central_heating | ONE_YEAR_FREE | 16482623 | n | n |
| Hessen | 110 | self_contained_central_heating | ONE_YEAR_FREE | 15462469 | n | n |

| picturecount | pricetrend | telekomuploadSpeed | totalRent | telekomTrackingGroup | telekomInternetTechnology | yearConstructed |
|---|---|---|---|---|---|---|
| 8 | 3.33 | 40 MBit/s | 440 | telekom_layer_magenta_l | A4ber VDSL | 1903 |
| 8 | 2.35 | 40 MBit/s | 1550 | telekom_layer_magenta_l | A4ber VDSL | 1900 |
| 10 | 5.10 | 10 MBit/s | 3130 | telekom_layer_magenta_m | A4ber VDSL | 1900 |
| 11 | 7.23 | 40 MBit/s | 920 | telekom_layer_magenta_l | A4ber Glasfaser | 2017 |
| 11 | 6.45 | 40 MBit/s | 600 | telekom_layer_magenta_l | A4ber VDSL | 2008 |
| 4 | 3.85 | 10 MBit/s | 310 | telekom_layer_magenta_m | A4ber VDSL | 1906 |

## Common Errors

Duplicated observations - Rows or records that appear more than once in a dataset, often due to data entry errors or merging issues, which can skew results by over-representing certain data points

Variables that are not defined well – Variables whose meaning, units, or scope are ambiguous or inconsistent, making them unreliable for analysis

Missing values - Absent or null data points within a dataset where a value was expected, which can introduce bias or errors if not handled properly

# Removing Redundant Attributes and Observations

## Removing Redundant Attributes

Once you have selected a set of attributes that you want to keep, you can use the „`select()`" command from the `tidyverse` package to select the attributes you want to keep.

> If 10% (or more) of the observations in a variable are missing, remove the variable.

```
> library(tidyverse)
> df <- read.csv("Dataset_Teachers.csv")
> head(df)
> summary(df)
```

```
> df.1 <- df %>%
  select(regio1,
         balcony,
         cld,
         regio2,
         totalRent,
         yearConstructed,
         noRooms,
         cellar,
         hasKitchen)
```

## Removing Redundant Observations

Duplicated observations can be easily removed using the command „`unique(object)`". When used on a dataframe, the `unique` command removes all observations that are identical.

In some occasions, it is possible that you have several observations from the same entity/person. Even though this may sometimes be wanted, you may want to only have a single observation of an individual in others. In these scenarios, the command „`unique()`" can be used as well.

> If 5% (or less) of the observations in a variable are missing, remove the observations.

```
> df.1 <- unique(df.1)
```

```
> lds.unique <- unique(df.1$ld)
```

```
...
```

# Dealing with Missing Values

R stores missing values as *NA*.

| 1 | 2 | 3 |
|---|---|---|

### 1. Detecting missing values

Function `"is.na(object name)"` — Input: a data frame, vector or matrix. Output: Categorical data frame, vector or matrix (True if the corresponding element in the object is missing and False, if the corresponding element in the object is not missing)

### 2. Removing missing values

### 3. Estimating missing values

```
> is.na(df.1)
> colSums(!is.na(df.1)) # Counts how many non-missing entries there are in each column
> colSums((is.na(df.1))) # Counts how many values are missing in each one of the attributes.
```

Output of `is.na(df.1)` — showing TRUE where values are missing (e.g. `yearConstructed` in rows 11, 12, 13, 14, 16, 24, 30, 32, 33, 38, 40):

| | regio1 | balcony | regio2 | totalRent | yearConstructed | noRooms | cellar | haskitchen |
|---|---|---|---|---|---|---|---|---|
| [1,] | FALSE | FALSE | FALSE | FALSE | FALSE | FALSE | FALSE | FALSE |
| [2,] | FALSE | FALSE | FALSE | FALSE | FALSE | FALSE | FALSE | FALSE |
| [11,] | FALSE | FALSE | FALSE | FALSE | TRUE | FALSE | FALSE | FALSE |
| [12,] | FALSE | FALSE | FALSE | FALSE | TRUE | FALSE | FALSE | FALSE |
| [16,] | FALSE | FALSE | FALSE | FALSE | TRUE | FALSE | FALSE | FALSE |
| [24,] | FALSE | FALSE | FALSE | FALSE | TRUE | FALSE | FALSE | FALSE |
| [30,] | FALSE | FALSE | FALSE | FALSE | TRUE | FALSE | FALSE | FALSE |
| [40,] | FALSE | FALSE | FALSE | FALSE | TRUE | FALSE | FALSE | FALSE |

# Dealing with Missing Values (cont.)

R stores missing values as *NA*.

1. **Detecting missing values.**

2. **Removing missing values.**

   The function `"complete()"` returns only complete rows of a dataframe. You can also remove all observations that do not have an entry in a certain attribute.

3. **Estimating missing values**

   We have not yet covered the most common estimation methods (regression and classification). We will show you how to implement them at a later point.

   Mean or Median.

## Code: Removing Missing Values

```
> is.na(df.1)
> df.1 <- na.omit(df.1)
> df.1 <- df.1 %>%
  filter(!is.na(df.1$yearConstructed))
```

If between 5% and 10% of the observations in a variable are missing, you will need to think about whether you really need the variable.

# Dealing with Outliers

Outliers are observations showing values outside the expected range, or data that is substantially different from the rest

**"Mechanical" approaches to identify outliers**

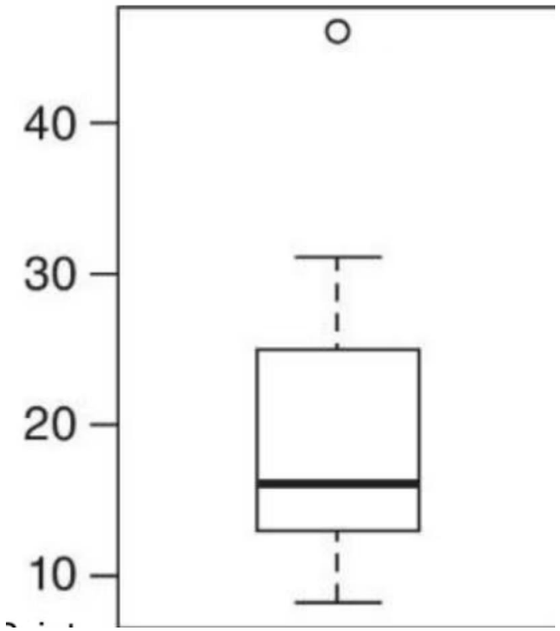
1. Observations outside e.g. *1.5 * \{InterquartileRange}*
2. Observations outside e.g. *3 * {Standard Deviation}*
3. Visualization can be used to detect unusual observations
4. And many others, like finite mixture models etc.

**You should not, however, just automatically remove those cases.**

1. Remember the normal distribution: 99.74% of cases should lie within *3 * {standard deviations}*
2. If you have 70.000 observations, you would expect around 182 "outliers", that are in fact perfectly good data

**Outliers can be produced by different causes**

1. Bad data / errors
2. True outliers, like the heaviest baby ever born with 10.2kg (average in Austria 2017: 3.317kg; 0.9% heavier than 4.5kg)




**Detecting Outliers – Option 1**

1. Compute first and third quartile, *Q_1* and *Q_3*
2. Compute interquartile range (*IQR*)
3. Compute boundary points
4. Check which observations have values outside the boundaries.

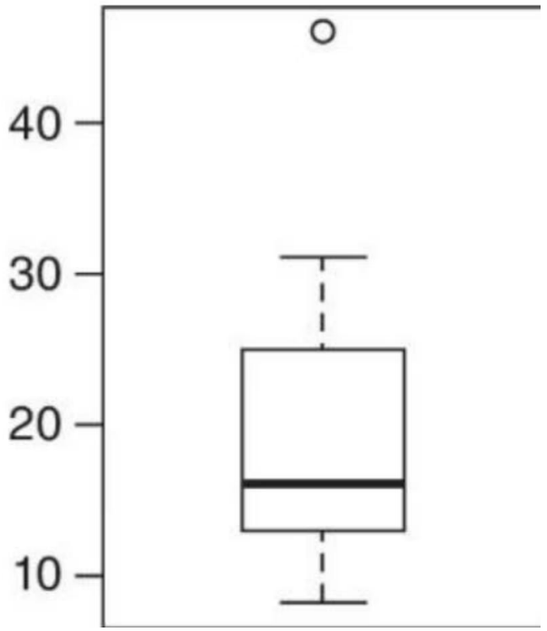
```
# Computing quartiles
> quartiles <- quantile(df.1$totalRent)
> Q.1 <- quartiles[2]
> Q.3 <- quartiles[4]

# Computing Inter Quartile Range
> inter.quartile.range <- IQR(df.1$totalRent)

# Computing Boundary Points
> lower.bound <- Q.1-1.5*inter.quartile.range
> upper.bound <- Q.3+1.5*inter.quartile.range

# Check for outliers
> df.1.outliers.below <- df.1$totalRent < lower.bound
> df.1.outliers.above <- df.1$totalRent > upper.bound
```

# Dealing with Outliers – Detection (Option 2) and Removal

### Detecting Outliers – Option 2

1. Normalize your dataset by computing z-scores.
2. Compute mean/median of the attribute.
3. Compute standard deviation of the attribute
4. Compute boundary points (mean/median $\pm 3$ SD)
5. Check which observations have values outside the boundaries.

### 2. Removing Outliers
### 3. Truncating Outliers




A box plot with a y-axis ranging from 10 to 40. The box plot has a median line at approximately 15, a box from approximately 12 to 25, and whiskers extending to approximately 10 and 30. A single outlier is plotted as a circle above the upper whisker at approximately 45.

```
# Normalize Dataset by computing z-scores
df.1$scaledRent <- scale(df.1$totalRent)

# Computing mean and standard deviation
> mean.rent <- mean(df.1$scaledRent)
> sd.rent <- sd(df.1$scaledRent)

# Computing Boundary Points
> lower.bound <- rent - 3 * sd.rent
> upper.bound <- rent + 3 * sd.rent

# Check for outliers
> df.1.outliers.below <- df.1$totalRent < lower.bound
> df.1.outliers.above <- df.1$totalRent > upper.bound
```

# Dealing with Outliers – Removing & Truncating

Outliers are easy to remove, but all exclusion of data can change your results and should be done with care. You should investigate the cases you exclude (with the method depending on the number of potential outliers). You should check the robustness of your results with regard to removal of outliers.

## 2. Removing Outliers

Removing outliers can be done by using the `"filter()"` function from the dplyr package.

```
# Removing outliers
> df.1 <- df.1 %>%
  filter(!(df.1$totalRent > upper.bound)) %>%
  filter(!(df.1$totalRent < lower.bound))
```

## 3. Truncating Outliers

Luckily, outliers are fairly easy to truncate as well. Truncating outliers means that we replace the observed variable values with the upper/lower bound found earlier.

```
# Truncating Outliers
> df.1$totalRent[(df.1$totalRent < lower.bound)] <-
lower.bound
> df.1$totalRent[(df.1$totalRent > upper.bound)] <-
upper.bound
```

# Questions?